

Computational Physics

Lecture 6: Linear algebra and matrices

Methods for solving Linear systems of equations:

- Gaussian elimination
- LU decomposition
- Inverse and determinant of a matrix
- Tri- and band-diagonal systems

Eigenvalue problem

- leading eigenvector
- calculating all eigenvalues/eigenvectors

Reference: *Computational Physics* by Mark Newman, Chapter 6

Linear system of equations

Generic problem: solve numerically a system of linear equations.

Let's start with the example of 4 simultaneous equations for w, x, y, z

$$2w + x + 4y + z = -4,$$

$$3w + 4x - y - z = 3,$$

$$w - 4x + y + 5z = 9,$$

$$2w - 2x + y + 3z = 7.$$

or in matrix form

$$\begin{pmatrix} 2 & 1 & 4 & 1 \\ 3 & 4 & -1 & -1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -4 \\ 3 \\ 9 \\ 7 \end{pmatrix}$$

Which then can be written in general as $\mathbf{Ax} = \mathbf{v}$ with $\mathbf{x} = (w, x, y, z)$

One impractical way to get the solution is to find the inverse of $\mathbf{A}$ and get $\mathbf{x} = \mathbf{A}^{-1}\mathbf{v}$

Let's see other faster and more practical ways

Linear equations: Gaussian elimination

Gaussian elimination is the base procedure for solving systems of linear equations

It is based on iterative transformation of the system of linear equations which preserve the solution (the solution stays the same)

The goal is to eliminate all entries in matrix **A** below the main diagonal

$$\begin{pmatrix} 2 & 1 & 4 & 1 \\ 3 & 4 & -1 & -1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -4 \\ 3 \\ 9 \\ 7 \end{pmatrix} \quad \longrightarrow \quad \begin{pmatrix} 1 & 0.5 & 2 & 0.5 \\ 0 & 1 & -2.8 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -2 \\ 3.6 \\ -2 \\ 1 \end{pmatrix} \quad \text{upper triangular}$$

To do so, we can apply the following two operations which preserve the solution:

1. multiply any row by a non-zero number
2. subtract from a given row any other row (with any non-zero factor)

Linear equations: Gaussian elimination

Let's do the Gaussian elimination for the example:

$$\begin{pmatrix} 2 & 1 & 4 & 1 \\ 3 & 4 & -1 & -1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -4 \\ 3 \\ 9 \\ 7 \end{pmatrix}$$

1) Make top-left element 1 by dividing 1st row by 2

$$\begin{pmatrix} 1 & 0.5 & 2 & 0.5 \\ 3 & 4 & -1 & -1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -2 \\ 3 \\ 9 \\ 7 \end{pmatrix}$$

2) make 1st element of 2nd row equal to 0, by subtracting 1st row times 3 from 2nd row

$$\begin{pmatrix} 1 & 0.5 & 2 & 0.5 \\ 0 & 2.5 & -7 & -2.5 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -2 \\ 9 \\ 9 \\ 7 \end{pmatrix}$$

3) make 1st element of 3rd and 4th row 0 using again subtracting from 1st row

$$\begin{pmatrix} 1 & 0.5 & 2 & 0.5 \\ 0 & 2.5 & -7 & -2.5 \\ 0 & -4.5 & 1 & 4.5 \\ 0 & -3 & -3 & 2 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -2 \\ 9 \\ 11 \\ 11 \end{pmatrix}$$

Finished modifying 1st column

Linear equations: Gaussian elimination

4) make 2nd element of 2nd row 1 by dividing row by 2

$$\begin{pmatrix} 1 & 0.5 & 2 & 0.5 \\ 0 & 1 & -2.8 & -1 \\ 0 & -4.5 & 1 & 4.5 \\ 0 & -3 & -3 & 2 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -2 \\ 3.6 \\ 11 \\ 11 \end{pmatrix}$$

5) make 2nd element of 3rd and 4th row 0 by subtraction using the 2nd row

Finished working on 2nd column

$$\begin{pmatrix} 1 & 0.5 & 2 & 0.5 \\ 0 & 1 & -2.8 & -1 \\ 0 & 0 & -13.6 & 0 \\ 0 & 0 & -11.4 & -1 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -2 \\ 3.6 \\ 27.2 \\ 21.8 \end{pmatrix}$$

6) same procedure to 3rd and 4th column

Final upper triangular matrix

$$\begin{pmatrix} 1 & 0.5 & 2 & 0.5 \\ 0 & 1 & -2.8 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -2 \\ 3.6 \\ -2 \\ 1 \end{pmatrix}$$

With such matrix it becomes quite simply to determine the solution with **backsubstitution**

Linear equations: Backsubstitution

After the Gaussian elimination we have the following system of equations

$$\begin{pmatrix} 1 & a_{01} & a_{02} & a_{03} \\ 0 & 1 & a_{12} & a_{13} \\ 0 & 0 & 1 & a_{23} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{pmatrix} \quad \text{or} \quad \begin{aligned} w + a_{01}x + a_{02}y + a_{03}z &= v_0, \\ x + a_{12}y + a_{13}z &= v_1, \\ y + a_{23}z &= v_2, \\ z &= v_3. \end{aligned}$$

The solution now proceeds through backsubstitution, i.e. we go from z to w

$$\begin{aligned} z &= v_3, \\ y &= v_2 - a_{23}z, \\ x &= v_1 - a_{12}y - a_{13}z, \\ w &= v_0 - a_{01}x - a_{02}y - a_{03}z. \end{aligned}$$

In our example gives $w = 2, \quad x = -1, \quad y = -2, \quad z = 1.$

Implementation of Gaussian elimination plus backsubstitution

Implement the Gaussian elimination and backsubstitution for a generic matrix A and apply to the example

$$\begin{pmatrix} 2 & 1 & 4 & 1 \\ 3 & 4 & -1 & -1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = \begin{pmatrix} -4 \\ 3 \\ 9 \\ 7 \end{pmatrix}$$

```
import numpy as np

def linsolve_gaussian(A, v):
    A = np.array(A, dtype=float, copy=True)
    v = np.array(v, dtype=float, copy=True)
    N = len(v)

    # Gaussian elimination
    for m in range(N):
        div = A[m, m]
        A[m, :] /= div
        v[m] /= div
        for i in range(m + 1, N):
            mult = A[i, m]
            A[i, :] -= mult * A[m, :]
            v[i] -= mult * v[m]

    # Backsubstitution
    x = np.empty(N, float)
    for m in range(N - 1, -1, -1):
        x[m] = v[m]
        for i in range(m + 1, N):
            x[m] -= A[m, i] * x[i]
    return x
```

```
A = np.array([[ 2,  1,  4,  1 ],
               [ 3,  4, -1, -1 ],
               [ 1, -4,  1,  5 ],
               [ 2, -2,  1,  3 ]], float)
v = np.array([-4, 3, 9, 7], float)

x = linsolve_gaussian(A, v)
print("x =", x)
```

```
x = [ 2. -1. -2.  1.]
```

Implementation of Gaussian elimination+backsubstitution

The trivial implementation of Gaussian elimination will fail if any of the diagonal elements becomes equal to zero in the process of solving

$$\begin{pmatrix} 0 & 1 & 4 & 1 \\ 3 & 4 & -1 & -1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -4 \\ 3 \\ 9 \\ 7 \end{pmatrix}$$

```
A = np.array([[ 0,  1,  4,  1 ],
              [ 3,  4, -1, -1 ],
              [ 1, -4,  1,  5 ],
              [ 2, -2,  1,  3 ]], float)

v = np.array([-4, 3, 9, 7], float)

x = linsolve_gaussian(A, v)
print(x)
```

[nan nan nan nan]

implementation fails because trying to divide with 0 in the first step
Better add a warning and stop program

```
# Gaussian elimination
for m in range(N):
    div = A[m, m]
    if div == 0:
        raise ZeroDivisionError(
            "Diagonal element is zero! Cannot solve the system with simple Gaussian elimination"
        )
```

Linear equations: Pivoting

In cases where we get 0 as diagonal element, use **pivoting**:

interchange two rows of the matrix

In our example, simply exchange rows 1 & 2 and then carry on with Gaussian elimination

$$\begin{pmatrix} 0 & 1 & 4 & 1 \\ 3 & 4 & -1 & -1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -4 \\ 3 \\ 9 \\ 7 \end{pmatrix} \quad \longrightarrow \quad \begin{pmatrix} 3 & 4 & -1 & -1 \\ 0 & 1 & 4 & 1 \\ 1 & -4 & 1 & 5 \\ 2 & -2 & 1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 3 \\ -4 \\ 9 \\ 7 \end{pmatrix}$$

Not obvious in general which equation to swap without introducing other problems.

Good scheme is **partial pivoting**:

before dividing by a diagonal element, always consider rearranging that row with lower ones.

Suppose you are at m^{th} row. Compare to all lower rows and look at row with m^{th} elements farthest from zero and swap the m^{th} row with that one.

Implementation of pivoting over Gaussian elimination and backsubstitution

Take older code with Gaussian elimination + backsubstitution and add pivoting

```
import numpy as np

def linsolve_gaussian(A, v):
    A = np.array(A, dtype=float, copy=True)
    v = np.array(v, dtype=float, copy=True)
    N = len(v)

    for m in range(N):
        #take row with largest element m
        #np.argmax gives index of maximum arg in subvector
        pivot_row = m + np.argmax(np.abs(A[m:, m]))
        if pivot_row != m:
            #produces two rows m and pivot_row and exchange them
            A[m, pivot_row] = A[pivot_row, m]
            v[m, pivot_row] = v[pivot_row, m]

        div = A[m, m]
        if abs(div) < 1e-15:
            raise ZeroDivisionError(
                "Matrix is singular (or nearly singular): no usable pivot"
            )

        A[m, :] /= div
        v[m] /= div

        for i in range(m + 1, N):
            mult = A[i, m]
            A[i, :] -= mult * A[m, :]
            v[i] -= mult * v[m]

    x = np.empty(N, float)
    for m in range(N - 1, -1, -1):
        x[m] = v[m]
        for i in range(m + 1, N):
            x[m] -= A[m, i] * x[i]
    return x
```

Linear equations: LU decomposition

Suppose now we have to solve many different sets $\mathbf{Ax} = \mathbf{v}$ with the same $\mathbf{A}$ and different $\mathbf{v}$

It would be wasteful to perform Gaussian elimination over and over, as the effect will be the same upper-triangular form on the left.

More efficient if we memorize all operations performed on $\mathbf{A}$ and somehow apply on $\mathbf{v}$:

LU decomposition

Start from

$$A = \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix} \quad (\text{indexed as in Python})$$

and perform same operation of Gaussian elimination, but expressed in matrix form

Linear equations: LU decomposition

For instance, to reduce first row to 1 0 0 0, instead of performing a set of operations we can do a single matrix multiplication

$$\frac{1}{a_{00}} \begin{pmatrix} 1 & 0 & 0 & 0 \\ -a_{10} & a_{00} & 0 & 0 \\ -a_{20} & 0 & a_{00} & 0 \\ -a_{30} & 0 & 0 & a_{00} \end{pmatrix} \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} 1 & b_{01} & b_{02} & b_{03} \\ 0 & b_{11} & b_{12} & b_{13} \\ 0 & b_{21} & b_{22} & b_{23} \\ 0 & b_{31} & b_{32} & b_{33} \end{pmatrix}$$

This matrix is also lower triangular:

$$\mathbf{L}_0 = \frac{1}{a_{00}} \begin{pmatrix} 1 & 0 & 0 & 0 \\ -a_{10} & a_{00} & 0 & 0 \\ -a_{20} & 0 & a_{00} & 0 \\ -a_{30} & 0 & 0 & a_{00} \end{pmatrix}$$

Linear equations: LU decomposition

The next step would be to modify the second row, again with a matrix multiplication:

$$\frac{1}{b_{11}} \begin{pmatrix} b_{11} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -b_{21} & b_{11} & 0 \\ 0 & -b_{31} & 0 & b_{11} \end{pmatrix} \begin{pmatrix} 1 & b_{01} & b_{02} & b_{03} \\ 0 & b_{11} & b_{12} & b_{13} \\ 0 & b_{21} & b_{22} & b_{23} \\ 0 & b_{31} & b_{32} & b_{33} \end{pmatrix} = \begin{pmatrix} 1 & c_{01} & c_{02} & c_{03} \\ 0 & 1 & c_{12} & c_{13} \\ 0 & 0 & c_{22} & c_{23} \\ 0 & 0 & c_{32} & c_{33} \end{pmatrix}$$

with a lower triangular matrix again :

$$\mathbf{L}_1 = \frac{1}{b_{11}} \begin{pmatrix} b_{11} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -b_{21} & b_{11} & 0 \\ 0 & -b_{31} & 0 & b_{11} \end{pmatrix}$$

And so on:

$$\mathbf{L}_2 = \frac{1}{c_{22}} \begin{pmatrix} c_{22} & 0 & 0 & 0 \\ 0 & c_{22} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -c_{32} & c_{22} \end{pmatrix}, \quad \mathbf{L}_3 = \frac{1}{d_{33}} \begin{pmatrix} d_{33} & 0 & 0 & 0 \\ 0 & d_{33} & 0 & 0 \\ 0 & 0 & d_{33} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Linear equations: LU decomposition

Putting the whole process together, the Gaussian elimination reduces to matrix multiplications

$$\mathbf{L}_3\mathbf{L}_2\mathbf{L}_1\mathbf{L}_0\mathbf{A}\mathbf{x} = \mathbf{L}_3\mathbf{L}_2\mathbf{L}_1\mathbf{L}_0\mathbf{v}$$

In practice calculations are done in a more convenient form: define the matrices

$$\mathbf{L} = \mathbf{L}_0^{-1}\mathbf{L}_1^{-1}\mathbf{L}_2^{-1}\mathbf{L}_3^{-1} \qquad \mathbf{U} = \mathbf{L}_3\mathbf{L}_2\mathbf{L}_1\mathbf{L}_0\mathbf{A}$$

and observe that $\mathbf{LU} = \mathbf{A}$ so that we can write directly

$$\mathbf{Ax} = \mathbf{v} \longrightarrow \mathbf{LUx} = \mathbf{v}$$

The advantage is that:

U is upper triangular by construct (final result of Gaussian elimination)

L is lower triangular (next slide)

Hence the name LU decomposition: we effectively decomposed A into the product of a lower triangular and an upper triangular matrices.

Linear equations: LU decomposition

E.g. the inverse of $\mathbf{L}_0 = \frac{1}{a_{00}} \begin{pmatrix} 1 & 0 & 0 & 0 \\ -a_{10} & a_{00} & 0 & 0 \\ -a_{20} & 0 & a_{00} & 0 \\ -a_{30} & 0 & 0 & a_{00} \end{pmatrix}$ is $\mathbf{L}_0^{-1} = \begin{pmatrix} a_{00} & 0 & 0 & 0 \\ a_{10} & 1 & 0 & 0 \\ a_{20} & 0 & 1 & 0 \\ a_{30} & 0 & 0 & 1 \end{pmatrix}$

and so on $\mathbf{L}_1^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & b_{11} & 0 & 0 \\ 0 & b_{21} & 1 & 0 \\ 0 & b_{31} & 0 & 1 \end{pmatrix}, \mathbf{L}_2^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & c_{22} & 0 \\ 0 & 0 & c_{32} & 1 \end{pmatrix}, \mathbf{L}_3^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & d_{33} \end{pmatrix}$

Giving $\mathbf{L} = \mathbf{L}_0^{-1} \mathbf{L}_1^{-1} \mathbf{L}_2^{-1} \mathbf{L}_3^{-1} = \begin{pmatrix} a_{00} & 0 & 0 & 0 \\ a_{10} & b_{11} & 0 & 0 \\ a_{20} & b_{21} & c_{22} & 0 \\ a_{30} & b_{31} & c_{32} & d_{33} \end{pmatrix}$

In practice, while performing the Gaussian elimination on $\mathbf{A}$, we also extract the appropriate elements of the matrix $\mathbf{L}$

Linear equations: LU decomposition

How LU decomposition is used in practice to solve $\mathbf{Ax} = \mathbf{v}$

1. Compute $\mathbf{U}$ (with set of matrix multiplications) and $\mathbf{L}$ (saving from matrix multiplication specific elements), obtaining the system $\mathbf{LUx} = \mathbf{v}$
2. Define a new vector $\mathbf{y} = \mathbf{Ux}$ so that the eq. becomes $\mathbf{Ly} = \mathbf{v}$
3. Solve the equation $\mathbf{Ly} = \mathbf{v}$ for the vector $\mathbf{y}$ using backsubstitution (remember $\mathbf{L}$ is triangular)
4. Solve $\mathbf{y} = \mathbf{Ux}$ in the variable $\mathbf{x}$ using backsubstitution (again $\mathbf{U}$ is triangular)

LU decomposition scales in terms of number of operations as $O(N^3)$, with N number of elements, while backsubstitution only as $O(N^2)$

Linear equations: LU decomposition

For example, for a 3x3 matrix:

First decompose $\mathbf{A} = \begin{pmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{pmatrix} = \mathbf{L}\mathbf{U} = \begin{pmatrix} l_{00} & 0 & 0 \\ l_{10} & l_{11} & 0 \\ l_{20} & l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} u_{00} & u_{01} & u_{02} \\ 0 & u_{11} & u_{12} \\ 0 & 0 & u_{22} \end{pmatrix}$

Obtaining the system $\mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{v}$ $\begin{pmatrix} l_{00} & 0 & 0 \\ l_{10} & l_{11} & 0 \\ l_{20} & l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} u_{00} & u_{01} & u_{02} \\ 0 & u_{11} & u_{12} \\ 0 & 0 & u_{22} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix}$

Define $\mathbf{y} = \mathbf{U}\mathbf{x}$ $\begin{pmatrix} u_{00} & u_{01} & u_{02} \\ 0 & u_{11} & u_{12} \\ 0 & 0 & u_{22} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ y_2 \end{pmatrix}$

Solve in $\mathbf{y}$ with backsubstitution $\mathbf{L}\mathbf{y} = \mathbf{v}$ $\begin{pmatrix} l_{00} & 0 & 0 \\ l_{10} & l_{11} & 0 \\ l_{20} & l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} v_0 \\ v_1 \\ v_2 \end{pmatrix} \longrightarrow \begin{aligned} y_0 &= \frac{v_0}{l_{00}}, \\ y_1 &= \frac{v_1 - l_{10}y_0}{l_{11}}, \\ y_2 &= \frac{v_2 - l_{20}y_0 - l_{21}y_1}{l_{22}}. \end{aligned}$

Solve in $\mathbf{x}$ with backsubstitution $\mathbf{y} = \mathbf{U}\mathbf{x}$

Linear equations: LU decomposition

Again, to avoid the problem of zero diagonal element, we should also add pivoting to this method

This method is already implemented e.g. in numpy or scipy

Typically, these implementations are very optimized and preferable:

```
import numpy as np

A = np.array([[ 0,  1,  4,  1 ],
              [ 3,  4, -1, -1 ],
              [ 1, -4,  1,  5 ],
              [ 2, -2,  1,  3 ]], float)
v = np.array([-4, 3, 9, 7], float)

x = np.linalg.solve(A, v)
print("x =", x)

x = [ 1.61904762 -0.42857143 -1.23809524  1.38095238]
```

Matrix inversion

Suppose we still want to compute inverse of $\mathbf{A}$, $\mathbf{A}^{-1}$. Applying Cramer's rule is still not a good idea.

We can turn the problem in solving a set of linear equations of the type

$$\mathbf{AX} = \mathbf{V} \quad \text{where } \mathbf{X} \text{ and } \mathbf{V} \text{ are now } N \times N \text{ matrices}$$

Use LU decomposition to reduce $\mathbf{A}$ to upper-triangular form, then solve separately the system for each column $\mathbf{x}_k$ of $\mathbf{X}$ and the corresponding column $\mathbf{v}_k$ of $\mathbf{V}$, with $k=1\dots N$

If we set $\mathbf{V}=\mathbf{I}$, and solve the set of equations $\mathbf{AX} = \mathbf{I}$

by definition $\mathbf{X}$ becomes the inverse matrix $\mathbf{X} = \mathbf{A}^{-1}$ as it satisfies $\mathbf{AA}^{-1} = \mathbf{I}$.

Although one can use `np.linalg.solve`, there is a specific function that embeds everything:

`np.linalg.inv(A)`

Complexity: 1 LU-decomposition $O(N^3)$ + N backsubstitutions $O(N^3)$

Matrix determinant

We also get the determinant of A from the LU decomposition:

$$\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{U})$$

The advantage is that $\mathbf{L}$ and $\mathbf{U}$ are triangular: all diagonal elements are its eigenvalues, and the determinant is their product

$$\det(\mathbf{A}) = \prod_{i=1}^N L_{ii} U_{ii}$$

With pivoting we also track the sign change in determinant for every swap, so with S swaps:

$$\det(\mathbf{A}) = (-1)^S \prod_{i=1}^N L_{ii} U_{ii}$$

Linear equations: Tridiagonal systems

A special case in solving linear equations arise when the matrix **A** is tridiagonal, with non-zero values only on the diagonal and immediately above and below

$$\mathbf{A} = \begin{pmatrix} a_{00} & a_{01} & & 0 \\ a_{10} & a_{11} & a_{12} & \\ & a_{21} & a_{22} & a_{23} \\ 0 & & a_{32} & a_{33} & a_{34} \\ & & & a_{43} & a_{44} \end{pmatrix}$$

These cases occur frequently in physics when looking e.g. at nearest-neighbor interactions

Can be solved in linear time $O(N)$ with Gaussian elimination and backsubstitution (do not need entire procedure)

Linear equations: Tridiagonal systems

E.g. $\begin{pmatrix} 2 & 1 & 0 & 0 \\ 3 & 4 & -5 & 0 \\ 0 & -4 & 3 & 5 \\ 0 & 0 & 1 & 3 \end{pmatrix}$ $\xrightarrow{\text{divide 1st row by 2 and subtract 3 times 2nd row}}$ $\begin{pmatrix} 1 & 0.5 & 0 & 0 \\ 0 & 2.5 & -5 & 0 \\ 0 & -4 & 3 & 5 \\ 0 & 0 & 1 & 3 \end{pmatrix}$ $\xrightarrow{\text{divide 2nd row by 2.5 and subtract -4 times 2nd row}}$

$\begin{pmatrix} 1 & 0.5 & 0 & 0 \\ 0 & 1 & -2 & 0 \\ 0 & 0 & -5 & 5 \\ 0 & 0 & 1 & 3 \end{pmatrix}$ $\xrightarrow{\text{and so on...}}$ $\begin{pmatrix} 1 & 0.5 & 0 & 0 \\ 0 & 1 & -2 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

We need only 1 division and 1 subtraction per row, for at most two elements

Backsubstitution is also $O(N)$

$$\begin{pmatrix} 1 & a_{01} & 0 & 0 \\ 0 & 1 & a_{12} & 0 \\ 0 & 0 & 1 & a_{23} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{pmatrix} \xrightarrow{\text{tridiagonal matrix algorithm or Thomas algorithm}} \begin{aligned} x_3 &= v_3, \\ x_2 &= v_2 - a_{23}x_3, \\ x_1 &= v_1 - a_{12}x_2, \\ x_0 &= v_0 - a_{01}x_1. \end{aligned}$$

Linear equations: Band-diagonal systems

A similar methodology can be extended when the matrix is banded

Band-diagonal system: in each row

- at most m_{lower} non-zero elements to the left of the main diagonal
- at most m_{upper} non-zero elements to the right of the main diagonal

$$A = \begin{pmatrix} a_{00} & a_{01} & a_{02} & & & & 0 \\ a_{10} & a_{11} & a_{12} & a_{13} & & & \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & & \\ & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & \\ & & a_{42} & a_{43} & a_{44} & a_{45} & a_{46} \\ & 0 & & a_{53} & a_{54} & a_{55} & a_{56} \\ & & & & a_{64} & a_{65} & a_{66} \end{pmatrix}$$

Overall complexity: $O(N \times m_{\text{upper}} \times m_{\text{lower}})$

Example of tridiagonal matrix: vibration in a one-dimensional system

Take N identical masses in a row connected by identical linear springs (can represent a model for vibration of atoms in a solid).
Let's find horizontal collective vibration modes



Equations of motions

$$m\ddot{\xi}_1 = k(\xi_2 - \xi_1) + F_1,$$
$$m\ddot{\xi}_i = k(\xi_{i+1} - \xi_i) + k(\xi_{i-1} - \xi_i) + F_i, \quad \text{for } i=2\dots N-1$$
$$m\ddot{\xi}_N = k(\xi_{N-1} - \xi_N) + F_N$$

ξ_i displacement of i^{th} mass relative to its rest position

m mass, k spring constant

F_i external force imposed on i

Example of tridiagonal matrix: vibration in a one-dimensional system

Suppose we apply a harmonic force (e.g. a varying electrical field) only to the first mass, in the form:

$$F_1 = C e^{i\omega t}$$

(although it is complex, at end just take real part)

The result will be an oscillation with angular frequency ω , so the overall solution should be of the form

$$\xi_i(t) = x_i e^{i\omega t}$$

x_i is a complex number, where magnitude controls the amplitude of vibration and its phase the phase of vibration relative to the driving force

Example of tridiagonal matrix: vibration in a one-dimensional system

Substituting the solution in the equation of motion

$$\begin{aligned} -m\omega^2 x_1 &= k(x_2 - x_1) + C, \\ -m\omega^2 x_i &= k(x_{i+1} - x_i) + k(x_{i-1} - x_i), \quad \text{for } i=2\dots N-1 \\ -m\omega^2 x_N &= k(x_{N-1} - x_N). \end{aligned}$$

Defining $\alpha = 2k - m\omega^2$

$$\begin{aligned} (\alpha - k)x_1 - kx_2 &= C, \\ \alpha x_i - kx_{i-1} - kx_{i+1} &= 0, \\ (\alpha - k)x_N - kx_{N-1} &= 0. \end{aligned}$$

or in matrix form

$$\begin{pmatrix} (\alpha - k) & -k & & & \\ -k & \alpha & -k & & \\ & -k & \alpha & -k & \\ & & \ddots & \ddots & \ddots \\ & & & -k & \alpha & -k \\ & & & & -k & (\alpha - k) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-1} \\ x_N \end{pmatrix} = \begin{pmatrix} C \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix}$$

The equations are real (the phase is the same for all positions, so x_i is just the amplitude).

Solve the problem using the Gaussian elimination program, with $N=100$, $C=1$, $m=1$, $k=6$, $\omega=2$, $\alpha=8$

Try to simplify the code in order to be more efficient given we are using a tridiagonal matrix.

Compare the time taken to solve the system

Plot the amplitude x_i as a function of i

Example of tridiagonal matrix: vibration in a one-dimensional system

```
import numpy as np
import matplotlib.pyplot as plt
import time

def linsolve_tridiag(A, v):
    A = np.array(A, dtype=float, copy=True)
    v = np.array(v, dtype=float, copy=True)
    N = len(v)

    # Perform the Gaussian elimination
    for i in range(N - 1):
        # Divide row i by its diagonal element
        A[i, i + 1] /= A[i, i]
        v[i] /= A[i, i]
        # Now subtract it from the next row down
        A[i + 1, i + 1] -= A[i + 1, i] * A[i, i + 1]
        v[i + 1] -= A[i + 1, i] * v[i]
    # Divide the last element of v by the last diagonal element
    v[N - 1] /= A[N - 1, N - 1]

    # Backsubstitution
    x = np.empty(N, float)
    x[N - 1] = v[N - 1]
    for i in range(N - 2, -1, -1):
        x[i] = v[i] - A[i, i + 1] * x[i + 1]
    return x
```

```
# Constants
N = 100
C = 1.0
m = 1.0
k = 6.0
omega = 2.0
alpha = 2*k-m*omega*omega

# Set up the initial values of the arrays
A = np.zeros([N,N],float)
for i in range(N-1):
    A[i,i] = alpha
    A[i,i+1] = -k
    A[i+1,i] = -k

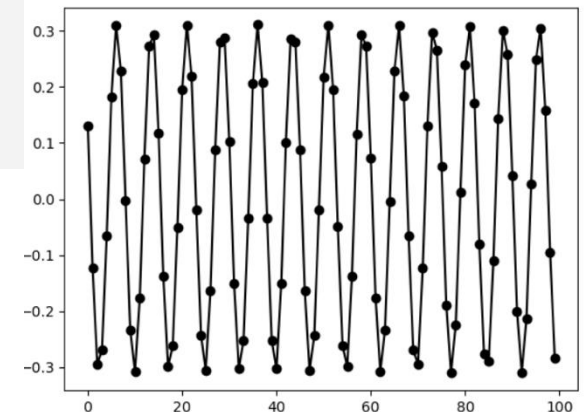
A[0,0] = alpha - k
A[N-1,N-1] = alpha - k
v = np.zeros(N,float)
v[0] = C

start = time.perf_counter()
x = linsolve_tridiag(A, v)
elapsed = time.perf_counter() - start
print(f"{elapsed:.6f} s")

# Make a plot using both dots and lines
plt.plot(x, "ko-")
plt.show()
```

In this case I am not performing operations over the entire matrix, but only on the two non zero elements.

We also saved time by not setting to zero to elements below the diagonal, which will not be used in backsubstitution



Eigenvalue problem

A common matrix problem in physics is the calculation of eigenvalues and eigenvectors of a matrix (e.g. classical and quantum mechanics).

The eigenvalue problem corresponds to the equation

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

with λ an eigenvalue and $\mathbf{v}$ the corresponding eigenvector

In most cases the matrix $\mathbf{A}$ is either real symmetric (or Hermitian when complex numbers are involved).

In these cases, for a $N \times N$ matrix there are N eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_N$ with real eigenvalues $\lambda_1, \dots, \lambda_N$.

The eigenvectors are orthogonal, and we assume to have them properly normalized, so $\mathbf{v}_i \cdot \mathbf{v}_j = \delta_{ij}$

Eigenvalue problem

General approach (not used in practice):

- Eigenvalues are roots of the characteristic polynomial

$$\det(\lambda I - A) = 0$$

- Once λ are found, the eigenvectors can be computed by solving linear system

$$(\mathbf{A} - \lambda \mathbf{I})\mathbf{v} = 0$$

This approach is impractical: numerically unstable to solve high-degree polynomial roots

Sometimes we only need the **eigenvector** with the corresponding **largest-magnitude eigenvalue**
e.g. when calculating ground state of a quantum system, or the dominant mode of a vibrating system

A way to compute it is the **power method**: multiply the matrix **A** into a trial vector until convergence

Eigenvalue problem: leading eigenvector

Start with any initial vector $\mathbf{x}_0$. It can be expressed in terms of the complete set of eigenvectors $\mathbf{v}_i$ of $\mathbf{A}$

$$\mathbf{x}_0 = \sum_{i=1}^N c_i \mathbf{v}_i$$

If we multiply by $\mathbf{A}$

$$\mathbf{x}_1 = \mathbf{A}\mathbf{x}_0 = \mathbf{A} \sum_{i=1}^N c_i \mathbf{v}_i = \sum_{i=1}^N c_i \mathbf{A}\mathbf{v}_i = \sum_{i=1}^N c_i \lambda_i \mathbf{v}_i$$

$\uparrow$
 $\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i.$

Multiply again by $\mathbf{A}$

$$\mathbf{x}_2 = \mathbf{A}\mathbf{x}_1 = \mathbf{A} \sum_{i=1}^N c_i \lambda_i \mathbf{v}_i = \sum_{i=1}^N c_i \lambda_i \mathbf{A}\mathbf{v}_i = \sum_{i=1}^N c_i \lambda_i^2 \mathbf{v}_i$$

After t multiplications:

$$\mathbf{x}_t = \mathbf{A}^t \mathbf{x}_0 = \sum_{i=1}^N c_i \lambda_i^t \mathbf{v}_i$$

Eigenvalue problem: leading eigenvector

Numbering eigenvalues from largest to smallest

$$|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_N|,$$

And dividing $\mathbf{x}_t$ by λ_1^t

$$\frac{\mathbf{x}_t}{\lambda_1^t} = \sum_{i=1}^N c_i \left(\frac{\lambda_i}{\lambda_1} \right)^t \mathbf{v}_i$$

As long as:

- 1) $\mathbf{x}_0$ is not orthogonal to $\mathbf{v}_1$
- 2) the leading eigenvalue λ_1 is unique

then $\left| \frac{\lambda_i}{\lambda_1} \right| < 1$ for any $i \neq 1$ and $\lim_{t \rightarrow \infty} \frac{\mathbf{x}_t}{\lambda_1^t} = c_1 \mathbf{v}_1$

$\mathbf{x}_t$ converges to a multiple of the leading eigenvector $\mathbf{v}_1$

Eigenvalue problem: leading eigenvector

Tips:

To avoid $\mathbf{x}_t$ growing too much, normalize it at each step $\mathbf{y}_t = A\mathbf{x}_t, \quad \mathbf{x}_{t+1} = \frac{\mathbf{y}_t}{\sqrt{\mathbf{y}_t \cdot \mathbf{y}_t}}$

Typically, 100 iterations are enough (wait for $\mathbf{x}_t$ to stop changing)

To compute eigenvalues, note that at convergence if you apply A $\mathbf{y}_t = A\mathbf{x}_t = \lambda_1 \mathbf{x}_t$

and so λ_1 can be obtained as $\mathbf{x}_t \cdot \mathbf{y}_t = \mathbf{x}_t \cdot A\mathbf{x}_t = \lambda_1 \mathbf{x}_t \cdot \mathbf{x}_t = \lambda_1$

A good choice for starting value $\mathbf{x}_0$ (to avoid it being orthogonal to $\mathbf{v}_1$) is either a random vector or a uniform one

$$\mathbf{x}_0 = \frac{(1, 1, \dots, 1)}{\sqrt{N}}$$

This method can be easily adapted to find the largest positive and negative eigenvalues, and the smallest (in magnitude), see book

Eigenvalue problem: calculating all eigenvectors with QR decomposition

Most widely used technique to compute all eigenvectors is **QR algorithm***

Define a $N \times N$ matrix of eigenvectors $\mathbf{V}$, where e.g. column k corresponds to the eigenvector $\mathbf{v}_k$

We can combine all equations $\mathbf{A}\mathbf{v}_i = \lambda\mathbf{v}_i$ into a single matrix equation

$$\mathbf{A}\mathbf{V} = \mathbf{V}\mathbf{D}$$

$\mathbf{D}$ is a diagonal matrix with entries corresponding to eigenvalues, $\mathbf{D} = \text{diag}(\lambda_1, \dots, \lambda_N)$.

Because the columns $\mathbf{v}_i$ of $\mathbf{V}$ are orthonormal vectors, $\mathbf{V}$ is an orthogonal matrix $\mathbf{V}^T = \mathbf{V}^{-1}$

*Has nothing to do with QR codes

Eigenvalue problem: calculating all eigenvectors with QR decomposition

QR decomposition: any real square matrix $\mathbf{A}$ permits a decomposition (we will not prove it here)

$$\mathbf{A} = \mathbf{Q}\mathbf{R}$$

where

- $\mathbf{Q}$ is orthogonal, $\mathbf{Q}^{-1}=\mathbf{Q}^T$, and thus, $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$
- $\mathbf{R}$ is upper diagonal

Let's assume we know how to compute it, and break $\mathbf{A}$ in its QR decomposition:

$$\mathbf{A} = \mathbf{Q}_1\mathbf{R}_1$$

Multiply by $\mathbf{Q}_1^T$

$$\mathbf{Q}_1^T\mathbf{A} = \mathbf{Q}_1^T\mathbf{Q}_1\mathbf{R}_1 = \mathbf{R}_1$$

Define a new matrix

$$\mathbf{A}_1 = \mathbf{R}_1\mathbf{Q}_1 \quad (\text{with product reversed}):$$

$$\mathbf{Q}_1^T\mathbf{A} = \mathbf{R}_1 \quad \longrightarrow \quad \mathbf{Q}_1^T\mathbf{A}\mathbf{Q} = \mathbf{R}_1\mathbf{Q} = \mathbf{A}_1$$

Eigenvalue problem: calculating all eigenvectors with QR decomposition

QR decompose $\mathbf{A}_1$ again and define $\mathbf{A}_2$ using the same procedure

$$\mathbf{A}_1 = \mathbf{Q}_2 \mathbf{R}_2, \quad \mathbf{A}_2 = \mathbf{R}_2 \mathbf{Q}_2 = \mathbf{Q}_2^T \mathbf{A}_1 \mathbf{Q}_2 = \mathbf{Q}_2^T \mathbf{Q}_1^T \mathbf{A} \mathbf{Q}_1 \mathbf{Q}_2$$

After k steps we get a sequence of matrices,
and it can be proved that for large enough k
 $\mathbf{A}_k$ becomes diagonal

$$\mathbf{A}_1 = \mathbf{Q}_1^T \mathbf{A} \mathbf{Q}_1$$

$$\mathbf{A}_2 = \mathbf{Q}_2^T \mathbf{Q}_1^T \mathbf{A} \mathbf{Q}_1 \mathbf{Q}_2$$

$$\mathbf{A}_3 = \mathbf{Q}_3^T \mathbf{Q}_2^T \mathbf{Q}_1^T \mathbf{A} \mathbf{Q}_1 \mathbf{Q}_2 \mathbf{Q}_3$$

...

$$\mathbf{A}_k = (\mathbf{Q}_k^T \cdots \mathbf{Q}_1^T) \mathbf{A} (\mathbf{Q}_1 \cdots \mathbf{Q}_k)$$

Suppose now $\mathbf{A}_k$ is diagonal, and define

$$\mathbf{V} = \mathbf{Q}_1 \mathbf{Q}_2 \mathbf{Q}_3 \cdots \mathbf{Q}_k = \prod_{i=1}^k \mathbf{Q}_i \quad \text{orthogonal because the } \mathbf{Q}_i \text{ are orthogonal}$$

$$\mathbf{D} = \mathbf{A}_k = \mathbf{V}^T \mathbf{A} \mathbf{V} \longrightarrow \mathbf{A} \mathbf{V} = \mathbf{V} \mathbf{D}$$

We arrive precisely at the definition of eigenvalues (matrix $\mathbf{D}=\mathbf{A}_k$) and eigenvectors (matrix $\mathbf{V}$)

Eigenvalue problem: calculating all eigenvectors with QR algorithm

QR algorithm to obtain eigenvalues and eigenvectors of matrix **A** based on an iterative procedure.

1. Initialize NxN matrix **V** holding eigenvectors, and set it to **I**
2. Calculates QR decomposition of **A**: **A = QR**
3. Compute **A** as **A = RQ** (no need to keep track of **A**)
4. Update **V** as **V = VQ**
5. Repeat from step 2 until **A** becomes diagonal, the matrix **D**

D will contain the eigenvalues and **V** the corresponding eigenvectors

Eigenvalue problem: calculating all eigenvectors with QR algorithm

Python implementation of QR algorithm

Calculate eigenvalues and eigenvectors of a real symmetric or Hermitian matrix: `np.linalg.eigh`

```
import numpy as np
#compute eigenvalues and eigenvectors
A = np.array([[1, 2],[2, 1]],float)
x,V = np.linalg.eigh(A)
print(x)
print(V)
#first and second eigenvector
print(V[:,0])
print(V[:,1])
```

$\lambda_1 \quad \lambda_2$
 $x = [-1. \quad 3.]$ eigenvalues

eigenvectors are in the columns: read as $V[:,i]$

$V = \begin{bmatrix} -0.70710678 & 0.70710678 \\ 0.70710678 & 0.70710678 \end{bmatrix}$
 $\downarrow \qquad \qquad \downarrow$
 $\mathbf{v}_1 \qquad \qquad \mathbf{v}_2$

Compute only eigenvalues (cheaper as we don't compute $\mathbf{V}$) for real symmetric or Hermitian matrix: `np.linalg.eigvalsh`

These algorithm assume the matrix is symmetric, so they don't even read the element above the diagonal. So if not symmetric, it still computes the same result (try!)

For non symmetric matrices use

`np.linalg.eig`

`np.linalg.eigvals`

QR algorithm via Gram–Schmidt process

Think of $\mathbf{A}$ as set of N column vectors

$$\mathbf{A} = \begin{pmatrix} | & | & | & \cdots \\ \mathbf{a}_0 & \mathbf{a}_1 & \mathbf{a}_2 & \cdots \\ | & | & | & \cdots \end{pmatrix}$$

Define new
vectors as

$$\mathbf{u}_0 = \mathbf{a}_0,$$

$$\mathbf{u}_1 = \mathbf{a}_1 - (\mathbf{q}_0 \cdot \mathbf{a}_1) \mathbf{q}_0,$$

$$\mathbf{u}_2 = \mathbf{a}_2 - (\mathbf{q}_0 \cdot \mathbf{a}_2) \mathbf{q}_0 - (\mathbf{q}_1 \cdot \mathbf{a}_2) \mathbf{q}_1,$$

...

$$\mathbf{u}_i = \mathbf{a}_i - \sum_{j=0}^{i-1} (\mathbf{q}_j \cdot \mathbf{a}_i) \mathbf{q}_j,$$

And normalized
counterparts

$$\mathbf{q}_0 = \frac{\mathbf{u}_0}{|\mathbf{u}_0|},$$

$$\mathbf{q}_1 = \frac{\mathbf{u}_1}{|\mathbf{u}_1|},$$

$$\mathbf{q}_2 = \frac{\mathbf{u}_2}{|\mathbf{u}_2|}.$$

...

$$\mathbf{q}_i = \frac{\mathbf{u}_i}{|\mathbf{u}_i|}.$$

The $\mathbf{q}_i$ are orthonormal

$$\mathbf{q}_i \cdot \mathbf{q}_j = \begin{cases} 1 & \text{se } i = j, \\ 0 & \text{se } i \neq j. \end{cases}$$

QR algorithm via Gram–Schmidt process

Original vectors can be expressed as:

$$\mathbf{a}_0 = |\mathbf{u}_0| \mathbf{q}_0,$$

$$\mathbf{a}_1 = |\mathbf{u}_1| \mathbf{q}_1 + (\mathbf{q}_0 \cdot \mathbf{a}_1) \mathbf{q}_0,$$

$$\mathbf{a}_2 = |\mathbf{u}_2| \mathbf{q}_2 + (\mathbf{q}_0 \cdot \mathbf{a}_2) \mathbf{q}_0 + (\mathbf{q}_1 \cdot \mathbf{a}_2) \mathbf{q}_1.$$

And $\mathbf{A}$ can be rearranged as a QR decomposition

$$\mathbf{A} = \begin{pmatrix} | & | & | & \cdots \\ \mathbf{a}_0 & \mathbf{a}_1 & \mathbf{a}_2 & \cdots \\ | & | & | & \cdots \end{pmatrix} = \underbrace{\begin{pmatrix} | & | & | & \cdots \\ \mathbf{q}_0 & \mathbf{q}_1 & \mathbf{q}_2 & \cdots \\ | & | & | & \cdots \end{pmatrix}}_{\mathbf{Q}} \underbrace{\begin{pmatrix} |\mathbf{u}_0| & \mathbf{q}_0 \cdot \mathbf{a}_1 & \mathbf{q}_0 \cdot \mathbf{a}_2 & \cdots \\ 0 & |\mathbf{u}_1| & \mathbf{q}_1 \cdot \mathbf{a}_2 & \cdots \\ 0 & 0 & |\mathbf{u}_2| & \cdots \end{pmatrix}}_{\mathbf{R}}$$

Implemented in `np.linalg.qr`